

1

EVOLUTION OF DISTRIBUTED CONTROL SYSTEMS

1.1. INTRODUCTION AND HISTORICAL BACKGROUND

The history of man's attempts to control industrial processes through automatic means is a long one (see Refs. 1.1 and 1.5), starting with such early developments as Cornelis Drebbel's furnace thermostat (1620) and James Watt's centrifugal governor for steam engines (1788). However, the major advances in integrated control system architectures, as compared to individual controllers, have taken place over the last fifty years. This section reviews several key developments during these years to provide the rationale for the recent emergence of the distributed control system architecture. The references at the end of the chapter provide additional historical detail.

Control systems have developed from the 1930s to the present day in response to two intertwined influences: user needs and technological advances. One factor that has influenced the control needs of the user is the continual growth in the size and complexity of industrial processes over the past fifty years. Also, the costs of raw materials and energy required to process these materials have increased substantially in this time. Finally, the labor costs involved in plant startup, operation, and maintenance have grown substantially. These influences have motivated the owners and operators of industrial processes to place a greater amount of emphasis on automation and on efforts to optimize their operations. In response to these user needs, the suppliers of industrial controls have been motivated to develop totally integrated-plant *management* systems that are more than the combination of individual control, monitoring, and data logging systems.

Fortunately, the explosive advances in technology that have taken place over the past fifty years have provided the capabilities needed for the evolution of such plant management systems. For example, the development of transistors, integrated analog circuits, and solid-state relays resulted in a growth in capability and an increase in reliability of electronic

control systems that enabled them to largely replace pneumatic control systems. Similarly, the development of digital technology in the form of improved large-scale integrated logic circuits, microprocessors, semiconductor memories, and cathode-ray tube (CRT) displays has led to even more impressive improvements in digital control system capabilities. These improvements have allowed control systems based on digital technology to replace electronic analog systems in many applications. References 1.2, 1.3, 1.4, and 1.6 trace the history of many of these technological developments.

The lines of technological development can be divided into two separate streams, as illustrated in Figure 1.1. The upper stream with its two branches is the more traditional one, and includes the evolution of analog controllers and other discrete devices such as relay logic and motor controllers. The second stream is a more recent one that includes the use of large-scale digital computers and their mini and micro descendants in industrial process control. These streams have merged into the current mainstream of distributed digital control systems. The dates of several key milestones in this evolutionary process are shown in Table 1.1 to illustrate the pace of these advances.

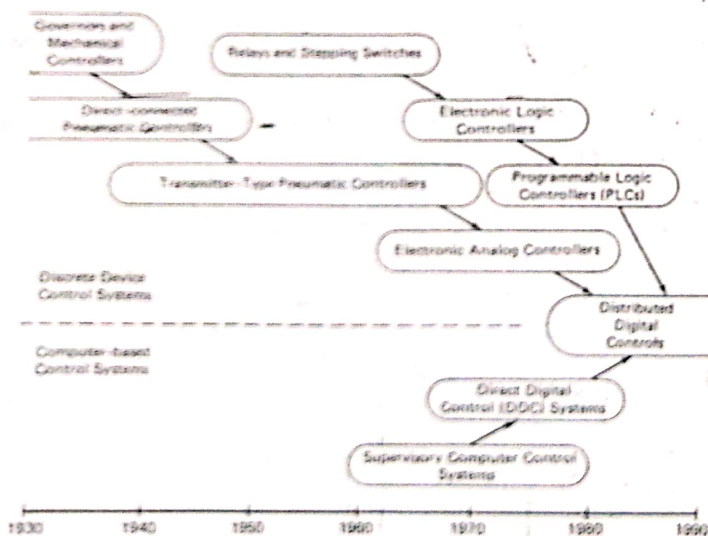


Figure 1.1. Evolution of Industrial Control Technology.

Table 1.1. Key Milestones in Control System Evolution.

1934	Direct-connected pneumatic controls dominate market.
1938	Transmitter-type pneumatic control systems emerge, making centralized control rooms possible.
1958	First computer monitoring in electric utility.
1959	First supervisory computer in refinery.
1960	First solid-state electronic controllers on market.
1963	First direct digital control (DDC) system installed.
1970	First programmable logic controllers (PLCs) on market.
1970	Sales of electronic controllers surpass pneumatic.
1975	First distributed digital control system on market.

1.1.1. Traditional Control System Developments

The concept of distributed control systems is not a new one. In fact, the early discrete device control systems listed in Figure 1.1 were distributed around the plant. Individual control devices such as governors and mechanical controllers were located at the process equipment to be controlled. Local readouts of set points and control outputs were available, and a means for changing the control mode from manual to automatic (or vice versa) usually was provided. It was up to the operator (actually, several operators) to coordinate the control of the many devices that made up the total process. They did this by roaming around the plant and making corrections to the control devices as needed and using the "Hey, Joe!" method of communications to integrate plant operations. This was a feasible approach to the control of early industrial processes because the plants were small geographically and the processes were not too large or complex. The same architecture was copied when direct-connected pneumatic controllers were developed in the late 1920s. These controllers provided more flexibility in selection and adjustment of the control algorithms, but all of the elements of the control loop (sensor, controller, operator interface, and output actuator) were still located in the field. There was no mechanism for communication between controllers other than that provided by each operator to other operators in the plant using visual and vocal means.

This situation changed of necessity in the late 1930s due to the growth in size and complexity of the processes to be controlled. It became more and more difficult to run a plant using the isolated-loop control architecture

described above. The emphasis on improving overall plant operations led to a movement towards centralized control and equipment rooms. This was made possible by the development of transmitter-type pneumatic systems. In this architecture (measurements made at the process, were converted to pneumatic signals at standard levels, which were then transmitted to the central location. The required control signals were computed at this location, then transmitted back to the actuating devices at the process. The great advantage of this architecture was that all of the process information was available to the operator at the central location. Thus, the operator was able to make better control decisions and operate the plant with a greater degree of safety and economic return.

The centralized control structure described above is still the dominant one in plants operating today. In the late 1950s and early 1960s, the technology used to implement this architecture started to shift from pneumatics to electronics. One of the key objectives of this shift was replacing the long runs of tubing used in pneumatic systems with the wires used in electronic ones. This change reduced the cost of installing the control systems and also eliminated the time lag inherent in pneumatic systems. Both of these advantages became more significant as plant sizes increased. Another consequence of the centralized control architecture was the development of the split controller structure. In this type of controller, the operator display section of the controller is panel mounted in the control room and the computing section is located in a separate rack in an adjoining equipment room. The split controller structure is especially appropriate for complex, interactive control systems (e.g., for boiler controls) in which the number of computing elements greatly exceeds the number of operator display elements.

Both pneumatic and electronic versions of the centralized control architecture still exist and are being sold, delivered, and operated today. In fact, it was not until 1970 that the sales of electronic controllers exceeded the sales of pneumatic controllers in the industrial process control marketplace (1.4).

The discussion to this point has focused on continuous, or analog, control devices, in which both the inputs and outputs to the controllers vary continuously over a selected range (e.g., 1-5 volts or 3-15 psi). Similar developments have taken place in the realm of sequential logic control devices, in which the inputs and outputs to the controllers take on only one of two discrete states (e.g., On/Off, 0/24 volts). These devices generally are used in controlling certain types of pumps, motors, or valves in a process. They also are used in safety override systems that operate in parallel to and back up the continuous systems described above. The original versions of these logic systems were implemented using simple elec-

tronic devices such as relays and stepping switches. Later, the development of solid-state electronic modules allowed logic systems to be implemented using the same level of technology as the corresponding electronic analog controllers.

In the early 1970s, a sophisticated device known as the programmable logic controller (PLC) was developed to implement sequential logic systems. This device is significant because it was one of the first special-purpose, computer-based devices that could be used by someone who was not a computer specialist. It was designed to be programmed by a user who was familiar with relay logic diagrams but was not necessarily a computer programmer. This approach to control system configuration was inspired by early efforts of process computer specialists to develop a process-oriented control language. However, it was more successful than most of these efforts in eliminating the user's dependence on a priesthood of computer specialists in running a process control system.

All of the versions of sequential logic systems described above have been implemented in direct-connected distributed architectures as well as in centralized ones. In each case, the logic controller has been associated directly with the corresponding unit of process equipment, with little or no communication between it and other logic controllers. It was not until the late 1970s that PLCs and computers started to be connected together in integrated systems for factory automation. For the purposes of this book, the PLC and networks of PLCs are considered to be special cases of the general distributed control system architecture described later in this chapter.

1.1.2. Computer-based Control System Developments

In addition to the evolution of the traditional types of control systems described above, a more recent (and equally important) evolution of computer-based process control systems has been taking place, as shown in the lower part of Figure 1.1. The first application of computers to industrial processes was in the areas of plant monitoring and supervisory control. In September 1958, the first industrial computer system for plant monitoring was installed at an electric utility power generating station (1.3). This innovation provided an automatic data acquisition capability not available before, and freed the operator from much drudgery by automatically logging plant operating conditions on a periodic basis. Shortly thereafter (in 1959 and 1960), supervisory computer control systems were installed in a refinery and in a chemical plant (1.4). In these applications, analog controllers were still the primary means of control. The computer used the available input data to calculate control set points that corre-

sponded to the most efficient plant operating conditions. These set points then were sent to the analog controllers, which performed the actual closed-loop control. The ability of supervisory control computers to perform economic optimization as well as to acquire, display, and log plant data provided the operator with a powerful tool for significantly improving plant operations.

The next step in the evolution of computer process control was the use of the computer in the primary control loop itself, in a mode usually known as direct digital control, or DDC. In this approach, process measurements are read by the computer directly, the computer calculates the proper control outputs, then sends the outputs directly to the actuation devices. The first DDC system was installed in 1963 in a petrochemical plant (1.3). For security, a backup analog control system was provided to ensure that the process could be run automatically in the event of a computer failure. This proved to be a wise precaution, because this early DDC installation (as well as many others) was plagued with computer hardware reliability problems. Despite these problems, it demonstrated many of the advantages digital control has over analog control: tuning parameters and set points do not drift, complex control algorithms can be implemented to improve plant operation, and control loop tuning parameters can be set adaptively to track changing operating conditions.

1.1.3. Resulting System Architectures

As a result of the developments described above, two industrial control system architectures came to dominate the scene by the end of the 1970s. While there are many variations, typical examples of these architectures are shown in Figures 1.2 and 1.3. The first architecture is a hybrid one, making use of a combination of discrete control hardware and computer hardware in a central location to implement the required control functions. In this approach, first level or local control of the plant unit operations is implemented by using discrete analog and sequential logic controllers (or PLCs). Panelboard instrumentation connected to these controllers is used for operator interfacing and is located in the central control room area. A supervisory computer and associated data acquisition system are used to implement the plant management functions, including operating point optimization, alarming, data logging, and historical data storage and retrieval. The computer also is used to drive its own operator interface, usually consisting of one or more video display units (VDUs). A substantial amount of interfacing hardware is required to tie the analog and sequential control equipment to each other as well as to the supervisory computer. The other dominant architecture, shown in Figure 1.3, is one in which

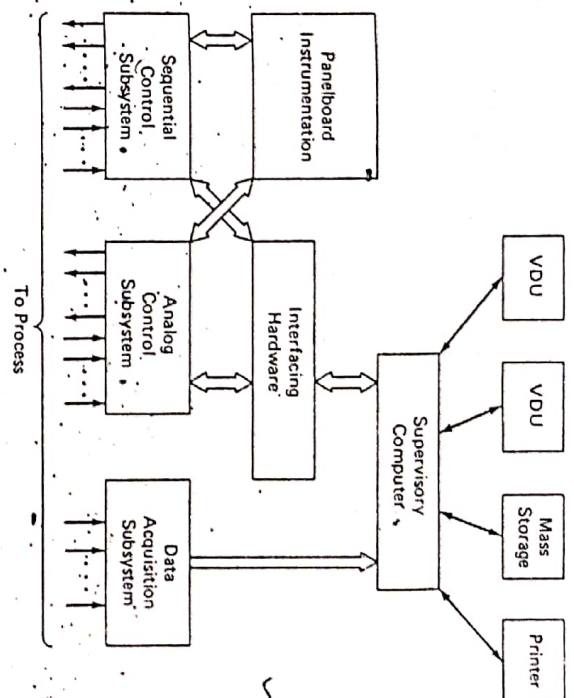


Figure 1.2. Hybrid System Architecture.

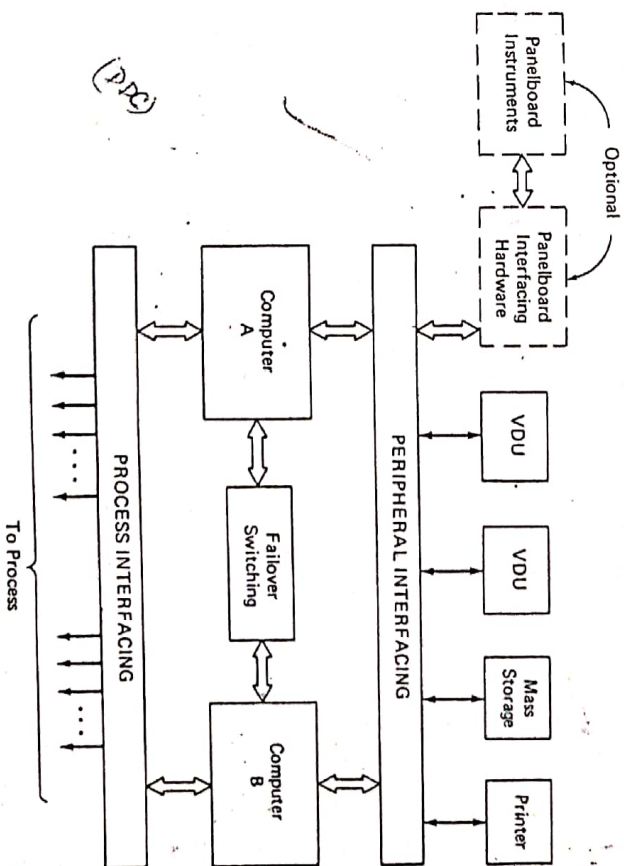


Figure 1.3. Central Computer System Architecture.

all system functions are implemented in high-performance computer hardware in a central location. In general, redundant computers are required so that the failure of a single computer does not shut the whole process down. Operator interfacing for plant management functions is provided using computer-driven VDUs, just as in the hybrid control system architecture described above. Operator interfacing for first-level continuous and sequential closed-loop control also may be implemented using VDUs. Optionally, the computers can be interfaced to standard panelboard instrumentation so that the operator in charge of first-level control can use a more familiar set of control and display hardware.

Note that both of the above systems use computers. The main difference between the two systems is the location of the implementation of the first-level continuous and sequential logic control functions. By the late 1970s, the hybrid system became by far the more prevalent approach in industrial control practice. The chemical and petroleum process industries heavily favored this approach, perhaps as a result of their disappointing experiences using early versions of direct digital control systems. In contrast, the use of large centralized computer systems to implement almost all plant control functions was limited primarily to the electric utility industry.

1.2. EMERGENCE OF THE DISTRIBUTED CONTROL SYSTEM ARCHITECTURE

While the central computer and hybrid system architectures provide significant advantages over earlier ones, they also suffer from a number of disadvantages. The biggest disadvantage of the centralized computer control architecture is that the central processing unit (CPU) represents a single point of failure that can shut down the entire process if it is lost. Since early industrial computer hardware was notoriously unreliable, two approaches were developed and have been used to attack the reliability problem: either a complete analog control system is used to back up the computer system, or another computer is used as a "hot standby" to take over if the primary control computer fails. Either approach results in a system significantly more expensive than an analog control system that performs a comparable set of functions. Another problem with these computer-based systems has been that the software required to implement all of the functions is extremely complex, and requires a priesthood of computer experts to develop the system, start it up, and keep it running. This is the natural result of an architecture in which a single CPU is required to perform a variety of functions in real time: input scanning, database updating, control algorithm computation, logging, long-term storage and retrieval of data, and man-machine interfacing (among others). Finally,

the centralized system is limited in its capability to accommodate change and expansion. Once the loading on the computer approaches its limit, it becomes very difficult to add on to the system without a significant decrease in performance or increase in cost.

The hybrid system architecture of Figure 1.2 also has its deficiencies. One of the worst is simply that it is composed of many different subsystems, often manufactured by different vendors. Just interfacing the subsystems to one another is a significant challenge, given the variety of different signal levels and conventions that exists in each. Starting them up and making them work as an integrated whole is no less difficult a task. The hybrid approach also is functionally limited compared to the central computer-based system. The benefits of digital control outlined in Section 1.1.2 are lost, since the closed-loop control is done by discrete analog and sequential devices. Also, the speed and accuracy of plant performance computations suffer due to the limitations of the analog input equipment and the problems in accessing the database, which is no longer centralized as in the computer implementation approach.

Because of these problems, it became clear to both users and system designers that a new architectural approach was needed. Control system engineers had been sketching out concepts of distributed systems composed of digital control and communication elements since the middle 1960s. Unfortunately, the technology to implement these concepts in a cost-effective manner was not available at that time. It was not until the microprocessor was introduced in 1971 that the distributed system architecture became practical. Supporting technology also became available during the early 1970s: inexpensive solid-state memories were developed to replace magnetic core memories; integrated circuit chips to implement standard communication protocols were introduced; display system technology flourished with the emergence of light-emitting diode (LED) and color CRT displays; in the software area, structured design techniques, modular software packages, and new on-line diagnostic concepts were developed.

The result of this fortunate confluence of user needs and technological developments was the introduction of a large number of distributed digital control system product lines by vendors in the late 1970s and early 1980s. (See Refs. 1.7-1.12 for tutorial information on distributed control systems and 1.13-1.26 for a review of the development of these systems.) While each system has a unique structure and specialized features, the architectures of most of these systems can be described in the context of the generalized one shown in Figure 1.4. The devices in this architecture are grouped into three categories: those that interface directly to the process to be controlled or monitored, those that perform high-level human in-

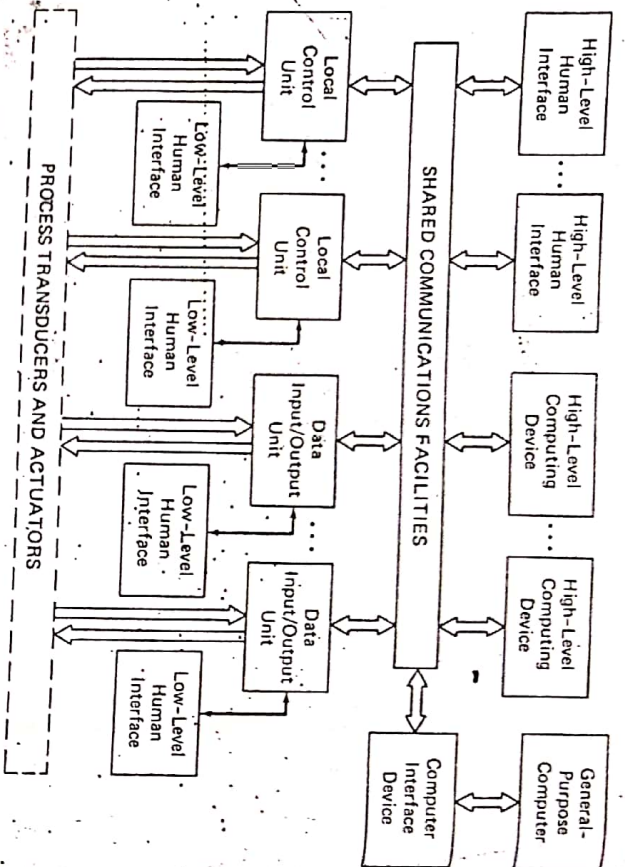


Figure 1.4. Generalized Distributed Control System Architecture.

terfacing and computing functions, and those that provide the means of communication between the other devices. A brief definition of each device is given below, and this terminology is used throughout the book: ✓

1. *Local control unit (LCU)*—The smallest collection of hardware in the system that can do closed-loop control. The LCU interfaces directly to the process.
2. *Low-level human interface (LLHI)*—A device that allows the operator or instrument engineer to interact with the local control unit (e.g., to change set points, control modes, control configurations, or tuning parameters) using a direct connection. LLHIs can also interface directly to the process. Operator-oriented hardware at this level is called a *low-level operator interface*; instrument engineer-oriented hardware is called a *low-level engineering interface*.
3. *Data input/output unit (DIOU)*—A device that interfaces to the process solely for the purpose of acquiring or outputting data. It performs no control functions.
4. *High-level human interface (HLHI)*—A collection of hardware that performs functions similar to the LLHI but with increased capability

- and user friendliness. It interfaces to other devices only over the shared communication facilities. Operator-oriented hardware at this level is called a *high-level operator interface*; instrument engineer-oriented hardware is called a *high-level engineering interface*.
5. *High-level computing device (HLCD)*—A collection of microprocessor-based hardware that performs *plant management functions* traditionally performed by a plant computer. It interfaces to other devices only over the shared communication facilities.
6. *Computer interface device (CID)*—A collection of hardware that allows an external general-purpose computer to interact with other devices in the distributed control system using the shared communication facilities.
7. *Shared communication facilities*—One or more levels of communication hardware and associated software that allow the sharing of data among all devices in the distributed system. Shared communication facilities do not include dedicated communication channels between specific devices, or between hardware elements within a device.

Not included in the architecture in Figure 1.4 but of vital importance to the design of a distributed control system are the packaging and electrical power systems.

Detailed descriptions of the seven distributed control system elements mentioned above and a discussion of the major issues involved in selecting, using, and designing these elements form the bulk of the remaining chapters in this book, as follows:

1. *Local control unit*—Architectural and hardware issues are discussed in Chapter 2; software and language issues are covered in Chapter 3; and security design issues are discussed in Chapter 4.
2. *Low-level human interface*—The low-level operator interface is discussed in Chapter 6; and the low-level engineering interface is covered in Chapter 7.
3. *Data input/output unit*—Many design issues overlap with local control unit discussions and are found in Chapters 2 and 3; specific process input/output design issues are covered in Chapter 4.
4. *High-level human interface*—The high-level operator interface is discussed in Chapter 6, while Chapter 7 covers engineering interface.
5. *High-level computing device*—Discussed in Chapter 8.
6. *Computer interface device*—Discussed in Chapter 8.
7. *Packaging and power systems*—Discussed in Chapter 8.

12. DISTRIBUTED CONTROL SYSTEMS

A brief discussion of future trends in the development of distributed control systems is provided in the last section of Chapter 8.

1.3. COMPARISON WITH PREVIOUS ARCHITECTURES

One of the main objectives in the development of distributed control systems has been to maintain the best features of the central computer control and hybrid architectures described in the previous section. Most importantly, the new systems have been structured to combine the power and flexibility of digital control with the user-oriented familiarity of the traditional analog and sequential control systems. A summary of some of the key features of distributed control systems compared to previous ones is given in Table 1.2, and additional information on the architectural advantages and disadvantages of distributed systems is provided in references (1.27-1.36). The following discussion of these features expands upon the table:

1. **Scalability and expandability**—Refers to the ease with which a system can be sized for a spectrum of applications, ranging from small to large, and the ease with which elements can be added to the system after initial installation. The hybrid system is quite modular, so it ranks high on both counts; the same holds for the distributed system architecture. On the other hand, the central computer architecture is designed for only a small range of applications. It is not cost-effective for applications much smaller than its design size and it cannot be expanded easily once its memory and performance limits are reached.
2. **Control capability**—Refers to the power and flexibility of the control algorithms that can be implemented by the system. The capability of the hybrid architecture is limited by the functions available in the hardware modules that make up the system. To add a function involves both adding hardware and rewiring the control system. On the other hand, central computer and distributed architectures both provide the full advantages of digital control: driftless set points and tuning parameters, availability of complex control algorithms, ability to change algorithms without changing hardware, remote and adaptive tuning capabilities, and many others.
3. **Operator interfacing capability**—Refers to the capability of the hardware provided to aid the operator in performing plant monitoring and control functions. The operator interface in the hybrid system consists of conventional panelboard instrumentation for normal control and monitoring functions and a separate video display unit.

Table 1.2. Comparison of Architectures.

FEATURE	HYBRID ARCHITECTURE	CENTRAL COMPUTER ARCHITECTURE	DISTRIBUTED ARCHITECTURE
1. Scalability and expandability	Good due to modularity	Poor—very limited range of system size	Good due to modularity
2. Control capability	Limited by analog and sequential control hardware	Full digital control capability	Full digital control capability
3. Operator interfacing capability	Limited by panelboard instrumentation	Digital hardware provides significant improvement for large systems	Digital hardware provides improvement for full range of system sizes
4. Integration of system functions	Poor due to variety of products	All functions performed by central computer	Functions integrated in a family of products
5. Significance of single-point failure	Low due to modularity	High	Low due to modularity
6. Installation costs	High due to discrete wiring and large volume of equipment	Medium—saves control room and equipment room space but uses discrete wiring	Low—savings in both wiring costs and equipment space
7. Maintainability	Poor—many module types, few diagnostics	Medium—requires highly trained computer maintenance personnel	Excellent—automatic diagnostic and module replacement

(VDU) for supervisory control. In the central computer and distributed architectures, VDUs generally are used as the primary operator interface for both the normal and supervisory control functions. The VDUs provide significant benefits to the operator: reduction in time needed to access control stations, flexibility of station grouping, graphics displays that mimic the process layout, and others. Since the VDUs in the distributed system are driven by microprocessors rather than by a large computer, they can be applied in a cost-effective way to small systems as well as large ones.

4. *Integration of system functions*—Refers to the degree with which the various functional subsystems are designed to work with one another in an integrated fashion. A high degree of integration minimizes user problems in procuring, interfacing, starting up, and maintaining the system. Since the hybrid system is composed of a variety of individual product lines, it is usually poorly integrated. The central computer architecture is well integrated because all of the functions are performed by the same hardware. The distributed system lies somewhere in between, depending on how well the products that make it up are designed to work together. (There are both good and bad examples of system integration out on the market today.)

5. *Significance of single-point failure*—Refers to the sensitivity of the system's performance to a failure of any of its elements. In the central computer architecture, the failure of a hardware element in the computer can cause the system to stop performing completely unless a backup computer is used. Therefore, this system is very sensitive to single-point failures. On the other hand, both the hybrid and distributed architectures are relatively insensitive to single-point failures due to the modularity of their structure.

6. *Installation costs*—Refers to the cost of system wiring and the cost of control room and equipment room space needed to house the system. The installation costs of the hybrid system are high: much custom wiring is needed for internal system interconnections; long wiring runs are needed from sensors to control cabinets; much control room space is required to house the panelboard instrumentation; and the large volume of control modules needed to implement the system take up a lot of equipment room space. The central computer architecture cuts down on this cost by eliminating the module interconnection wiring and by using VDUs to replace much of the panelboard instrumentation. The distributed system reduces costs further by using a communication system to replace the sensor wiring runs

and by reducing required equipment room space through the use of space-efficient microprocessor-based modules.

7. *Maintainability*—Refers to the ease with which a system can be kept running after installation. Low maintainability implies high maintenance costs, including the cost of spares, costs of process downtime while repairs are being made, and personnel training costs. The hybrid system is particularly poor in this area because of the large number of spare modules required, the lack of failure diagnostics in the system, and the personnel training required to cover the diverse subsystems. The central computer architecture is somewhat better: the range of module types is reduced and a certain number of failure diagnostics are provided. However, relatively sophisticated personnel are required to maintain the complex computer hardware and software. On the other hand, the maintainability of the distributed system architecture is excellent. Since there are only a few general-purpose control modules in the system, spare parts and personnel training requirements are minimal. Automatic on-line diagnostics are available to isolate failures to the module level, and module replacements can be made without disrupting a major portion of the process.

It can be seen from the table that the distributed control system architecture provides the user with many benefits over the hybrid and central computer architectures. The comparison is not all one-sided, however; as with any new venture, moving from a conventional analog control system to a distributed one requires the user to deal with a number of potential difficulties and changes in operation. One of the most obvious changes is that a microprocessor-based control system represents a new technology that plant personnel must learn. A certain amount of retraining of operating, instrument and maintenance people is required to ensure the success of any first installation of a distributed control system in a plant. Operating procedures will change; the operators will be spending a greater percentage of their time monitoring the process from the control room than patrolling the plant. When in the control room, they will be running the process from a video display unit instead of from panelboard instrumentation. During the early introduction of VDUs to the control room, the switch was expected to be traumatic for the operators. However, the transition turned out to be relatively painless; whether this was due to an underestimation of the adaptability of humans or to the pretraining effects of video games and home computers is not clear.

The new distributed systems offer the user a tremendous amount of flexibility in choice of control algorithms and location of equipment in the

Distributed Control Architecture and Applications Issues

- 1.27 Borsi, L., and Pavlik, E., "The Concepts and Structures of Distributed Process Automation Systems," *Process Automation*, no. 2, 1980, pp. 63-70.
- 1.28 Eriksson, L.L., and Purvis, J.R. III, "Centralized vs. Distributed Control: A Cost Comparison," *InTech*, vol. 28, no. 6, June 1981, pp. 61-63.
- 1.29 Natiello, L.F., "Distributed Digital Control: Practical Considerations for Design and Implementation," Third IFAC Workshop on Distributed Control Systems, Beijing, China, August 1981.
- 1.30 Garrett, L.T., and McHenry, J.M., "Analyzing Costs of Digital and Analog Control Systems," *Hydrocarbon Processing*, vol. 60, no. 12, December 1981, pp. 103-108.
- 1.31 Leahy, M.J., and Cornell, J.W., "A Discussion of Distributed Microprocessor-based Systems and Applications for Utility Boiler Control," 25th ISA Power Instrumentation Symposium, Phoenix, May 1982.
- 1.32 Stanton, B.D., "Reduce Problems in New Control System Design," *Hydrocarbon Processing*, vol. 61, no. 8, August 1982, pp. 67-70.
- 1.33 Damsker, D.J., "Controversial Issues of Distributed Control," *Proceedings of Joint Power Generation Conference*, Indianapolis, September 1983; IEEE, New York, 1984, pp. 522-529.
- 1.34 Hoffmann, W., "Problems in the Application of Digital Automation Systems in the Process Industry," *Process Automation*, no. 2, 1983, pp. 69-76.
- 1.35 Quetzsch, G., "Comments on the Structures of Digital Automation Systems," *Process Automation*, no. 2, 1983, pp. 77-82.
- 1.36 Damsker, D.J., chairman, "Application Guide, Distributed Control and Monitoring for Power Generating Stations," *IEEE Power Engineering Society Standard P-1046*, 1986.

Distributed Control System Evaluation and Procurement

- 37 Rapley, D.E., "A Practical Approach for Selection of a CRT-based Distributed Process Control/Data Acquisition System," *ISA Publication* ISBN 87664-432-9, 1979.
- 38 "Selecting Digital Control Systems," *InTech*, vol. 26, no. 5, May 1979, pp. 28-32.
- 39 Lynn, S.R., "Guidelines for Specifying and Procuring Distributed Control Systems," *InTech*, vol. 27, no. 9, September 1980, pp. 87-91.
- 40 Dobrowolski, M., "Guide to Selecting Distributed Digital Control Systems," *InTech*, vol. 28, no. 6, June 1981, pp. 45-52.
- 41 Cocheo, S., "How to Evaluate Distributed Computer Control Systems," *Hydrocarbon Processing*, vol. 60, no. 6, June 1981, pp. 95-106.
- 42 Pollock, W.K., "Evaluating Distributed Control Systems—A Generic Approach," *Proceedings of Third Annual Control Engineering Conference*, Control Engineering, May 1984, pp. 121-124.
- 43 Zimmerman, C.K., "Evaluating Distributed Control Systems: Where Do We Go From Here?" *Control Engineering*, vol. 31, no. 10, October 1984, pp. 109-112.
- 44 Cordova, G.J., Hertanu, H.I., and Doyle, G.T., "Microprocessor-based Distributed Control Systems," *Chemical Engineering*, vol. 92, no. 2, January 21, 1985, pp. 86-95.

2 LOCAL CONTROL UNIT ARCHITECTURE

2.1. INTRODUCTION

As defined in Chapter 1, the local control unit (LCU) is the smallest collection of hardware in the distributed control system that performs closed-loop control. That is, it takes inputs from process-measuring devices and commands from the operator and computes the control outputs needed to make the process follow the commands. It then sends the control outputs to the actuators, drives, valves, and other mechanical devices that regulate the flows, temperatures, pressures, and other variables to be controlled in the plant.)

Since an LCU malfunction can cause a condition that is hazardous to both people and equipment, its proper design is critical to the safe and efficient operation of the plant. The issues involved in evaluating or designing LCU architectures are discussed in this chapter. First, the basic elements of a microprocessor-based controller are defined. Then the concept of function blocks is introduced to allow discussion of LCU functionality in the context of a popular control language approach. Next, several LCU architectures prevalent in the distributed control marketplace are compared with respect to cost and performance features. Finally, a number of hardware design issues that affect LCU architecture are discussed. Other issues dealing with LCU languages, security, and input/output capabilities are covered in later chapters.)

2.2. BASIC ELEMENTS OF A MICROPROCESSOR-BASED CONTROLLER.

To provide a starting point for discussion, the basic elements of a generalized microprocessor-based LCU can be defined as shown in Figure 2.1. The microprocessor along with its associated clock comprise the central processing unit (CPU) of the controller. Read only memory (ROM) is used for permanent storage of controller programs, and random-access semiconductor memory (RAM) is used for temporary storage of infor-

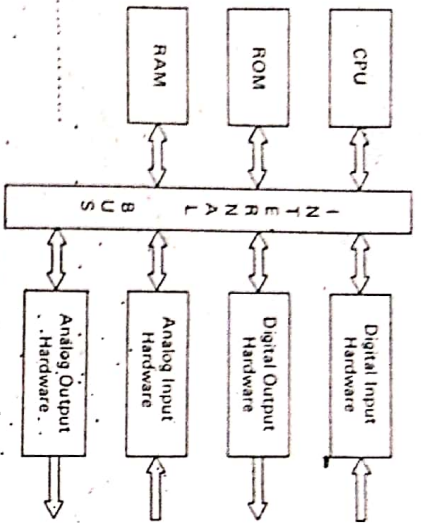


Figure 2.1. Basic Elements of a Local Control Unit.

maion. Depending on the type of microprocessor used, RAM and ROM can be located on the microprocessor chip or on separate memory chips. The LCU also must have input/output (I/O) circuitry so that it can communicate with the external world by reading in, or receiving, analog and digital data as well as sending similar signals out. Generally, the CPU communicates with the other elements in the LCU over an internal shared bus that transmits addressing, data control, and status information in addition to the data. A detailed description of these microprocessor system elements and how they work is beyond the scope of this book. For more information on the basic operation of digital system components and how they relate to industrial controllers, refer to References 2.1-2.14 at the end of the chapter.

The controller structure shown in Figure 2.1 is the minimum required to perform basic control functions. In a noncritical application in which the control function never changes, this structure might be adequate (e.g., in a home appliance whose failure would not cause a safety problem). The control algorithms could be coded in assembly language and loaded into ROM. After the controller was turned on, it would read inputs, execute the control algorithms, and generate control outputs in a fixed cycle indefinitely. However, because the situation is not this simple in industrial control applications, the controller structure shown in Figure 2.1 must be enhanced to include the following:

1. *Flexibility of changing the control configuration*—In industrial applications the same controller product usually is used to implement a great variety of different control strategies. Even for a particular

strategy, the user usually wants the flexibility of changing the control system tuning parameters without changing the controller hardware. Therefore, the control configuration cannot be burned into ROM but must be stored in a memory medium whose contents can be changed, such as RAM. Unfortunately, RAM is usually implemented using semiconductor technology that is volatile; that is, it loses its contents if the power is turned off (whether due to power failure, routine maintenance, or removal of the controller from its cabinet). Therefore, some provision must be made for restoring the control configuration, either from an external source or from a nonvolatile memory within the controller itself.

2. *Ability to use the controller without being a computer expert*—The typical user of an industrial control system is generally familiar with the process to be controlled, knows the basics of control system design, and has worked with electric analog or pneumatic control systems before. However, the user is usually not capable of or interested in programming a microprocessor in assembly language. He or she simply wants to be able to implement the selected control algorithms. Therefore, a mechanism for allowing the user to "configure" the LCU's control algorithms in a relatively simple way must be provided.

3. *Ability to bypass the controller in case it fails so that the process still can be controlled manually*—Shutting down the process is very expensive and undesirable for the control system user. Since all control equipment has the potential of failing no matter how carefully it has been designed, the system architecture must allow an operator to "take over" the control loop and run it by hand until the control hardware is repaired or replaced.

4. *Ability of the LCU to communicate with other LCUs and other elements in the system (as shown in the generalized system architecture in Figure 1.4)*—Controllers in an industrial control system do not operate in isolation but must work in conjunction with other controllers, data I/O devices, and human interface devices. A mechanism for allowing the LCU to perform this interaction must be provided.

The remainder of this chapter, as well as Chapters 3 and 4, will discuss the various ways these needs can be satisfied in designing an industrial-grade LCU.

2.3. FUNCTION BLOCKS—AN INTRODUCTION

As indicated above, the LCU must provide a means for entering the control algorithms to be executed without requiring the user to be a computer

Programmer. This is usually done by means of a control language supplied with the LCU. Chapter 3 gives a detailed discussion of the issues involved in selecting and evaluating control languages. However, a brief introduction to them is given here to facilitate discussion of LCU architectures.

The control language provided in commercially available industrial controllers usually is one of two types: a control-oriented version of a high-level language such as FORTRAN or BASIC, or a block-oriented language patterned after the control functions traditionally provided as discrete hardware modules in electronic or pneumatic control systems (e.g., summation, square root, or control modules). The discussion of the high-level language approach will be deferred to Chapter 3; this approach is generally used only in computer systems or in the most powerful microprocessor-based LCUs.

By far the more popular approach is to use function blocks to define the control system configuration to be implemented in the LCU. The concept of function blocks can be appreciated most easily through an example, such as that shown in Figure 2.2. In this example, the LCU has two analog inputs and one analog output. Also, the LCU provides the user with a library of function blocks in ROM, which may include proportional-integral-derivative (PID) control, square root, summation, difference, high/low select, and other continuous functions. Logical functions required for sequence control and safety systems also may be provided, such as AND, OR, NOT, time delay, and latching functions. To configure the control

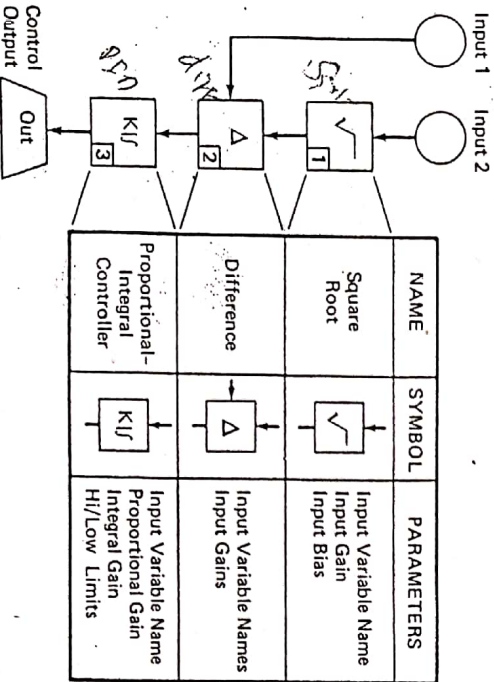


Figure 2.2. Continuous Control Example

logic shown in Figure 2.2, the user would first select the square-root function from the library and give it a block number, say 1 (as shown in the bottom right-hand corner). Then, the user would set the various parameters that are associated with the function. In the case of the square-root function, there are three: the source of input to the block (controller hardware input 2), and the gain and bias adjustments to the input signal required to convert it to the proper units in the control structure. Then, the user would select the difference block and identify it as block 2. The parameters associated with this block are the input sources (i.e., hardware input 1 and the output of function block 1) and the gains on the input signals. Next, the user selects the proportional-integral controller function block, labels it, and sets its parameters in a similar manner. The last step is to link the output of function block 3 (control output in the figure) to the physical output of the LCU.

The means by which this function block selection and configuration takes place is not significant here (this subject is covered in Chapter 7). The main point is that the function block concept allows the user not familiar with computer programming to configure and tune the control system. This can be done in a simple procedural fashion using many of the same steps carried out in the past with electronic analog and pneumatic control product lines. In this case, however, the function blocks are implemented in software modules within the LCU instead of as individual hardware modules that must be selected from a catalog and wired up into a control system.

(The function block approach has proved to be very successful in relieving the user from the need to learn a programming language in order to use a distributed control system.) In this chapter, one of the key parameters in the various control system architectures considered is the size of the control configuration (i.e., the number of function blocks) permitted per LCU. Other aspects of evaluating function blocks and selecting control languages will be discussed in Chapter 3.

2.4. A COMPARISON OF ARCHITECTURES

While Figure 2.1 describes the basic elements of all microprocessor-based local control units, the current offerings of controllers in the marketplace exhibit endless variations on this structure. The controllers differ in size, I/O capability, range of functions provided, and other architectural parameters depending on the application and the vendor who designed the equipment. The purpose of this section is to examine the impact of these architectural parameters on the performance, cost, and application flexibility of an LCU.

2.4.1. Architectural Parameters

When evaluating the controllers on the market or when specifying a new one, the control system designer is faced with the problem of choosing a controller architecture that best meets the needs of the range of applications in which the controller is to be used. Some of the major architectural parameters that must be selected include the following:

1. *Size of controller*—This refers to the number of function blocks and/or language statements that can be executed by the controller, as well as the number of process I/O channels provided by the controller.
2. *Functionality of controller*—This refers to the mix of function blocks or language statements provided by the controller (e.g., continuous control, logic control, arithmetic functions, or combinations of the above). This also refers to the mix of process input and output types (e.g., analog or digital) provided by the controller.
3. *Performance of controller*—This refers to the rate at which the controller scans inputs, processes function blocks or language statements, and generates outputs; it also includes the accuracy with which the controller performs these operations.
4. *Communication channels out of controller*—In addition to process inputs and output channels, the controller must provide other communication channels to operator interface devices and to other controllers and devices in the system. The number, type, and speed of these channels are key controller design parameters.
5. *Controller output security*—In a real-time process control system, a mechanism must be provided (usually manual backup or redundancy) to ensure that the control output is maintained despite a controller failure so that a process shutdown can be avoided.

Unfortunately, it is not generally possible to select any one of these architectural parameters independently from all of the others, since there is a great degree of interaction among them. Therefore, selecting the best combination for the range of applications to be considered is more a matter of engineering judgment than a science. Each vendor of microprocessor-based systems has a different view of the range of applications intended for the controller, and as a result designs an LCU architecture that quite often differs from that of its competitors.

To illustrate some of the differences in LCU architectures, three representative LCU configurations are shown in Figures 2.3 through 2.5. They are not intended to represent particular commercially available products but rather, different classes of controllers on the market today.

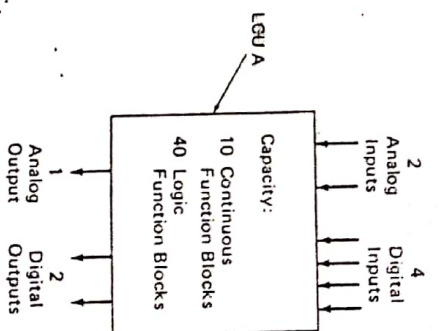


Figure 2.3. LCU Architecture—Configuration A

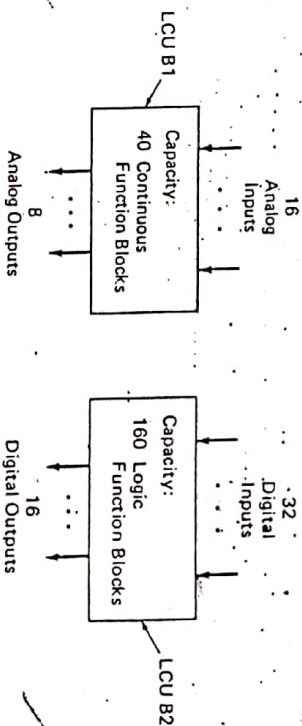


Figure 2.4. LCU Architecture—Configuration B

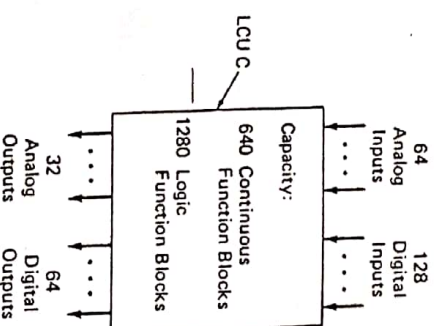


Figure 2.5. LCU Architecture—Configuration C

Configuration A (Figure 2.3) represents a class of single-loop LCU that provides both analog and digital inputs and outputs and executes both continuous and logic function blocks. Configuration B (Figure 2.4) represents an architecture in which two different types of LCUs implement the full range of required continuous and logic functions. Configuration C (Figure 2.5) represents a multiloop controller architecture in which both continuous and logic functions are performed. For each example, the architecture is defined in terms of its analog and digital I/O capacity and its continuous and logic function block capacity. In Table 2.1, these architectures are compared on the basis of the parameters defined previously: size, functionality, performance, communications, and output security.

Table 2.1. Comparison of Architectures

ARCHITECTURE PARAMETERS	CONFIGURATION A (SINGLE-LOOP)	CONFIGURATION B (2 LCU TYPES)	CONFIGURATION C (MULTI-LOOP)
Controller size	Number of functions needed for single PID loop or motor controller.	Includes functions and I/O needed for eight control loops and a small logic controller.	System size is equivalent to small DDC system.
Controller functionality	Uses both continuous and logic function blocks.	Continuous and logic function blocks split between controllers.	Uses both continuous and logic function blocks; can support high-level languages.
Controller scalability	High degree of scalability from small to large systems.	Requires both controller types even in small systems.	Not scalable to very small systems.
Controller performance	Requirements can be met with inexpensive hardware.	Because of functional split, performance requirements are not excessive.	Hardware must be high performance to execute large number of functions.
Communication channels	Need intermodule communications for control; only minimum needed for human interface.	Functional separation requires close interface between controller types.	Large communication requirement to human interface; minimal between controllers.
Controller output security	Controller has single-loop integrity; usually only manual backup is needed.	Lack of single-loop integrity requires redundancy in critical applications.	Size of controller requires redundancy in all applications.

provisions. The following discussion expands on this table for each of the architectures. Additional discussions in the areas of communication requirements and controller output security are given in Sections 4.1 and 4.2 of Chapter 4.

LCU Configuration A. In configuration A, the controller size is the minimum required to perform a single loop of control or a single motor control function or other simple sequencing function. Two digital outputs are provided to allow the controller to drive a pulsed (raise/lower) positioner or actuator. Twice as many inputs as outputs are provided to permit implementation of algorithms such as cascade control, temperature compensation of flows, and interlocking of logic inputs and continuous control loops. A general-purpose controller such as this requires both continuous and sequential (logic) function blocks to be included in its library. In most industrial control applications, the performance of the LCU must be adequate to sample all inputs, compute all of the function blocks, and generate all outputs in the range of 0.1 to 0.5 seconds maximum. Since configuration A has such a small number of inputs, function blocks, and outputs, the performance requirement can be met easily by a simple and inexpensive set of microprocessor-based hardware (e.g., an eight-bit microprocessor and matching memory components).

The communication requirements on configuration A for purposes of human interfacing are minimal because only one loop is controlled and a few input points monitored. However, it is important that a secure inter-controller communication channel be provided so that the single-loop controller can participate in complex control system structures containing other LCUs.

An important architectural feature to be considered when evaluating industrial controllers is the provision for control output security. In commercially available controllers, one or both of the following methods are used to allow continued operation of the process in the event of a controller failure: (1) a backup feature is provided to allow the operator to adjust the control output manually if the automatic controller fails; or (2) a redundant controller is provided to allow continuation of automatic control if the primary controller fails. The choice of method depends on the number of control outputs that would be lost if a controller fails. An operator can handle a small number of loops (usually in the range of one to four) manually, so for small controllers usually only a manual backup is provided. In the case of controller architecture A, the single-loop integrity of the controller configuration allows the simple and inexpensive option of manual backup to be used in most applications. For controllers that implement larger numbers of control loops, some form of control redundancy is usu-

ally provided. See Section 4.2 for a more extensive discussion of the issues regarding controller security.

LCU Configuration B. Configuration B differs significantly from configuration A. In the first place, two different types of LCUs (continuous control, or B1 in Figure 2.4, and logic control, or B2) are used to provide the full range of required controller functionality. In general, increasing the number of types of controllers in the system has both positive and negative effects. On the positive side, the specialized design of each controller allows it to match the functional needs of the corresponding application more closely than would a single general-purpose controller. As a result, a particular control application would require a smaller number of controller hardware modules. On the other hand, an increase in the number of controller types from one to two also results in increased interfacing requirements between controllers, additional documentation and training needs, and a decrease in production volume for each controller type (resulting in higher unit costs and longer lead times). In addition, this increase can reduce the scalability of the resulting system, since in the general case *both* of the controller types will be needed in even the smallest system. As usual, the optimum tradeoff is dependent on the range of applications foreseen for the system being designed.

With respect to controller size, configuration B is medium in scale. The continuous control portion has the form of an eight-loop controller, and the logic control portion can be viewed as a small programmable logic controller (PLC) or equivalent. Because of the split in functions and the relatively small size of each controller, the performance requirements on the controller hardware to meet the 0.1–0.5 second cycle time are not excessive. This class of controller usually is implemented using a high-performance eight-bit or an average performance 16-bit microprocessor and matching memory components.

In the area of communication channels required, configuration B calls for a well-designed interface between the two controller types, since in many systems both controllers must operate in close coordination to integrate continuous and logic control functions. The ability to communicate with other controllers in the distributed system must be provided, but the communication performance level need not be as high as in configuration A since a greater percentage of communications takes place within the LCU. Of course, as with Configuration A, a communication channel to human interface devices must be provided, but the channel must have a larger bandwidth because of the larger volume of traffic required per controller.

Finally, regarding output security, the lack of single-loop integrity of

configuration B implies that a redundant controller must be used in all critical applications (i.e., those in which a controller failure would cause a significant upset to the process). Of course, if the application requires both continuous and logic control, redundancy must be provided for both of the controller types in the system. It should be noted that full one-on-one redundancy often is not used in commercially available controllers in this class. Instead, one redundant controller may be used to back up several primary controllers. This reduces the cost of redundancy to some extent, but increases the complexity of the hardware used for interfacing the primary and backup controllers.

LCU Configuration C. LCU configuration C shown in Figure 2.5 is closer in structure to the direct digital control (DDC) systems described in Chapter 1 than are either of the other two configurations. It is designed to be a multiloop controller in which all functions are performed by one CPU in conjunction with its associated memory and I/O boards. This places stringent requirements on the performance of the hardware since all of the control algorithms in the LCU must be executed within 0.5 seconds or less. This LCU configuration is usually implemented with one or more 16-bit microprocessors or a 32-bit microprocessor in conjunction with support hardware such as arithmetic coprocessors to attain the required speed.

Since the objective of this architecture is to implement a large number of functions in one controller, usually both continuous and logic function blocks are provided. Because of the power of the microprocessor supplied in this configuration, it also becomes feasible to include a high-level language (usually BASIC or FORTRAN) in addition to or instead of function blocks.

This architecture has a number of advantages over configurations A and B. For example, requirements for communication among controllers are minimal because of the high density of functions implemented per controller; also, this LCU's high-level language capability allows it to implement complex user-defined control and computational algorithms. However, it is not as scalable to small systems as are the other two architectures, and the communication port or ports to the operator interface hardware must handle a large volume of traffic. In addition, since a failure in this type of LCU could affect a large number of control loops, a redundant CPU (and perhaps redundant I/O hardware as well) must be included in all control applications of any significance. Redundant hardware may not be required if only a few loops are affected by a failure or if the controller is performing only high-level control functions and is not manipulating control outputs directly.

The cost per loop for each configuration was computed by dividing the total cost of the required controllers by the number of loops (12 in the above example).

A number of conclusions can be drawn from the results in Figure 2.6. First, it can be seen that the cost-per-loop relationship for LCU A is a constant at 1,000 doubloons per loop; this is obviously as expected for a single-loop controller. The cost per loop for configuration B (LCU B1) is relatively high for a small number of loops, but decreases quickly as the number of loops increases and the full capacity of the controller is utilized. For 16 loops and beyond, the cost-per-loop of LCU B1 is less than that for LCU A. Since LCU C is designed to be a 32-loop controller, it does not become cost-effective until the application approaches the number of loops in that range. However, if the number of loops required goes above 32, it is again at a significant disadvantage compared to the other controllers, since additional hardware is required beyond that point. A final conclusion can be reached by inspection of Figure 2.6: if the choice of a controller type is based strictly on cost per loop, configuration A would be selected for small systems (under 16 loops) and configuration B would be selected for larger ones.

It should be noted, however, that this conclusion is valid only for the case of simple, single-loop control system configurations. To allow investigation of other configurations, the concept of the *control complexity ratio* (CCR) of a control system structure is defined as follows:

$$\text{CCR} = \frac{\text{Number of function blocks in control system}}{\text{Number of control system outputs}}$$

Because case I is concerned with continuous functions only, for this case:

$$\text{CCR} = \frac{\text{Number of continuous function blocks}}{\text{Number of analog outputs}}$$

The absolute value of this ratio will vary somewhat depending on the definition of a function block in the particular system considered (some function blocks are more complicated than others). However, for a given definition, this ratio provides a very useful index of control system complexity. For example, the simple control loop defined above has five function blocks per control output, so its CCR is five. If five more function blocks have to be added to this configuration to form a cascade controller, its CCR would then be 10. In more complicated control strategies such as those for a distillation column or an industrial boiler, control complexity

ratios may get as high as 30 or 40. It will become clear that the choice of the best or most cost-effective controller architecture depends a great deal on the complexity of the control system to be implemented in the controller.

To determine the effect of control system complexity on the cost-effectiveness of the various LCU configurations, an example control system having 100 control outputs was evaluated. The number of function blocks in the control system was varied to produce a range of CCR values from 1 to 20. For each value of CCR, the number of LCUs required to implement the corresponding control system then was computed for each LCU configuration defined in Figures 2.3 through 2.5. Finally, the cost per function block was determined by dividing the cost of the LCUs by the number of function blocks in the control system. The results of this analysis are plotted in Figure 2.7.

It is evident from the figure that the cost per function is very high for all LCU configurations for low values of complexity. In this range, the number of function blocks implemented is low, while the number of controllers required is fixed by the requirement to provide 100 control outputs. As the complexity of each control configuration increases, the cost per function block decreases until it reaches a minimum level. It is interesting to note that the complexity ratio at which each cost curve reaches its minimum value corresponds closely to the characteristic complexity ratio of the LCU configuration itself. For example, the cost curve for configuration B keeps decreasing until $\text{CCR} = 5$, at which point the curve remains flat. This value of CCR corresponds to the number of continuous

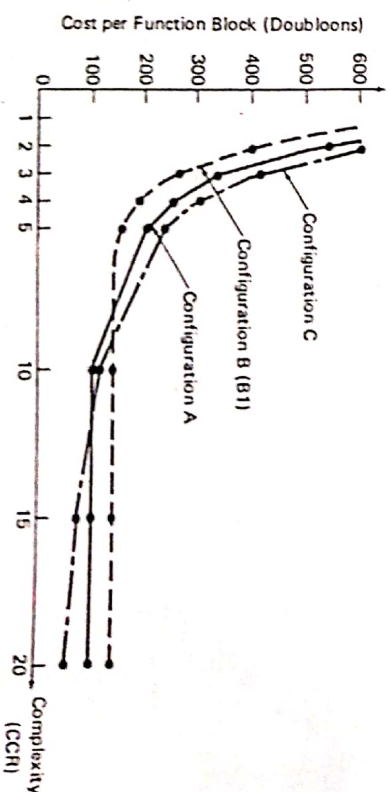


Figure 2.7. Cost-per-Function Relationship (Continuous Functions Only)

8. Communication functions to human interface devices that allow operators and engineers to interact with the LCU.

Another desirable feature is for the language to allow the user to modify these standard functions or to include custom functions within the control language structure.

As the LCUs become more powerful in processing capability, it is becoming feasible and cost-effective to distribute functions previously performed by a central computer into the LCUs. These functions include plant performance monitoring, long-term data storage and retrieval, and computing of optimal set points for plant operation. In the past, these functions have been implemented using a general purpose programming language, such as FORTRAN or BASIC, in the central computer. Therefore, it is essential that the LCUs be able to support these types of high-level languages so that the user can make use of software already in his or her program-library or available as standard commercial packages.

As much as possible, the control language selected should be transportable; that is, it should not be dependent on the microprocessor used in the LCU. There are two reasons for this requirement: (1) the user may want to implement the same control logic on systems provided by different vendors, and (2) the same control logic should be usable if the control system vendor decides to upgrade the LCU by going to a new, more powerful microprocessor.

Obviously, the control language must be compatible with the hardware environment in which it will run and with the performance requirements of the control system. The system must provide for sufficient memory capacity to store the control algorithms and data in the LCU. The microprocessor selected must be powerful enough to execute the control algorithms fast enough to meet the needs of the process to be controlled.

3.3. LANGUAGE ALTERNATIVES

While the cost of digital hardware has decreased dramatically over the last several years, the cost of developing and implementing the software portion of digital systems has increased significantly. This is especially true when software costs are evaluated as a percentage of total project costs. According to Boehm (3.1) and others, software development and maintenance now account for over 80% of the total life-cycle costs of a digital computer- or microprocessor-based system.

Because of this trend, the designers of industrial control systems have moved strongly in the direction of *productized software*, that is, software that provides the user with a set of prepackaged functions the user can *configure* (as opposed to *program*) to accomplish the computational or

control functions needed for the application. This approach limits the user's flexibility somewhat, but it means the user will be less dependent on software experts (either internal or external to the user's organization) to set up and maintain the control system. If the productized software is well designed for the application, it can offer the user all of the capabilities needed while eliminating a large percentage of the costs that a custom software design would incur.

✓ The major language alternatives currently available to the industrial control user are:

1. Function blocks, preprogrammed functions whose parameters can be set by the user and which can be linked together, much like traditional discrete hardware modules;
2. Problem-oriented languages (POLs) that are customized for a specific type of control application;
3. High-level languages that offer the user some degree of flexibility in designing his or her own control algorithms while maintaining compatibility with function blocks or POLs in the system.

These alternatives are described in more detail in the following sections. References 3.2-3.8 provide additional information on the various types of control languages that are available for use in industrial control systems.

3.4. FUNCTION BLOCKS

In current distributed control systems, the most prevalent prepackaged control language at the LCU level is one of function blocks. (See References 3.9-3.17 for additional information in this area.) In this language approach, the user receives a standard library of control and computational functions that are preprogrammed and stored in ROM. The user then configures the control system by selecting the appropriate blocks from the library, linking them together, and setting the various input and tuning parameters associated with each block. Table 3.1 gives one example of a library of function blocks used in a commercially available controller. It is clear from the example that many of the functions provided are very similar to those available using hardware modules in discrete modulating and sequential control systems. However, the functional capability of the software function blocks goes beyond that of hardware modules in several areas:

1. Functions are provided that would be very difficult or expensive to implement in hardware. Examples include the analog time-delay block (used in the control of processes that have transport delays),

Table 3.1. Example of a Function Block Library

COMPUTATIONAL FUNCTIONS		SIGNAL PROCESSING FUNCTIONS	
Sum—2 input/4 input		Integrator	
Multiply		Lead/lag	
Divide		Moving average	
Square root		Analog time delay	
y^x , e^x		High/low limit	
$\log x$, $\ln x$		Rate limit	
Trigonometric			
Generalized polynomial		SIGNAL STATUS FUNCTIONS	
Function generator		High/low alarm	
Two-dimensional interpolation		High/low select	
Matrix addition		Analog transfer	
Matrix multiplication		Digital transfer	
CONTROL FUNCTIONS		LOGIC FUNCTIONS	
PID control		AND	
Pulse positioner		OR	
Adapt block		Qualified OR	
Smith predictor		NOT	
General digital controller		Latch	
INTERMODULE COMMUNICATIONS		Digital timer	
Analog input (local/plant level)		Up/down counter	
Analog input list		Remote control latch	
Analog output (local/plant level)		Pulse rate counter	
Digital input (local/plant level)		OPERATOR COMMUNICATIONS	
Digital input list		Control station	
Digital output (local/plant level)		Indicator station	
		Cascade station	
		Ratio station	

- the adapt block (which allows tuning parameters in one function block to be adjusted from an externally computed or transmitted signal), and the more complex computational blocks (e.g., exponential functions and matrix operations).
- Since the function blocks are implemented in software instead of in hardware, one can easily alter the control configuration without changing any controller wiring or purchasing additional hardware (assuming that the controller's function block capacity has not been exceeded).
- The computational capability of the digital controller allows the conversion of all physical inputs to the system to engineering units (e.g., from millamps and volts to degrees Celsius or pressure in atmospheres). All of the control and computing operations can then be

carried out using engineering units, thereby eliminating the tedious scaling and conversion operations required in conventional analog control systems.

3.4.1. Example of Continuous Control

To introduce the function block concept, Figure 2.2 gave an example of how function blocks are used in configuring a control strategy. This example can be interpreted as the implementation of a simple flow control loop. Inputs 1 and 2 are the physical inputs to the LCU; input 1 is the flow set point from an external pressure transducer. To convert input 2 into a flow signal, the user selects the square-root function block from the library and identifies its input as coming from physical input 2. The user then enters appropriate values of the gain and bias parameters to accomplish the desired square-root function and convert the input signal to engineering units (say, flow in gallons per minute). Next, the user selects the difference block in order to generate an error signal between the actual flow and the flow set point. Again, the user identifies the inputs to the difference block and chooses the gain corresponding to input 1 so that the set point also is in units of gallons per minute. Using the output of the difference block as its input, the user then selects a proportional-integral (PI) block to perform the control function. The tuning parameters (proportional gain and integral gain) that are appropriate for the valve and process dynamics then are selected, and the high/low control limits (if any) are set. Once the PI block output is linked to the physical output from the LCU, the control configuration is complete.

This example illustrates the results of the control configuration process. Chapter 7 will discuss in detail the devices and techniques by which the user interacts with the LCU to perform these configuration functions.

Once the control configuration has been entered, the LCU can be put on-line so that it can start executing the algorithms in the configuration. In most systems, the function block parameters that do not affect the control configuration can be *tuned* (changed) while the LCU is on-line; however, the LCU generally must be taken off-line if the control configuration itself must be changed. (This is equivalent to rewriting an analog control system.)

3.4.2. Example of Logic Control

The example in the last section was for a continuous control system, one in which the control variables can take on one of many values over a continuous range. Control strategies involving Boolean logic functions also

4 LOCAL CONTROL UNIT—PROCESS INTERFACING ISSUES

4.1. INTRODUCTION

In Chapters 2 and 3, the local control unit was discussed as if it were a self-contained unit having no interfaces with other elements in the distributed control system. The block diagram in Figure 2.1 illustrates this; only process input and output hardware is shown. In a limited number of dedicated control situations this hardware may be all that is required. However, in the great majority of applications, the LCU must communicate with many other system elements as shown earlier in the generalized distributed control system structure in Figure 1.4.

Figure 4.1 shows a block diagram illustrating these other interfaces from the point of view of the LCU. This figure expands on the basic LCU elements in Figure 2.1 through the addition of interfaces to external communication facilities and to a low-level human interface device.

The communications interfaces (one or more) permit the LCU to interact with the rest of the distributed system to accomplish several functions:

1. To allow several LCUs to implement control strategies that are larger in scope than possible with a single LCU;
2. To allow transmission of process data to the higher-level system elements (e.g., human interface and computing devices);
3. To allow these higher-level elements to transmit information requests and control commands to the LCUs;
4. To allow two or more LCUs to act together as redundant controllers to perform the same control or computational functions;
5. To augment the I/O capacity of the LCU with that of data input/output units (DI/OUs) in the system.

The low-level human interface device and its associated interface hardware allow several important human interfacing functions to be accomplished through hardware that is connected directly to the LCU rather than over the shared communication facilities. These functions include:

LOCAL CONTROL UNIT—PROCESS INTERFACING ISSUES 75

1. Allowing the plant operator to control the process (e.g., select control set points and controller modes);
2. Allowing the operator to override the automatic equipment and control the process manually in case of a controller hardware failure or other system malfunction;
3. Allowing the plant instrumentation engineer to configure the control system logic and later tune the control system parameters.

Later chapters will cover many of the issues involved in implementing these interface functions. Chapter 5 will provide a detailed discussion of the facilities required for generalized communications between LCUs and other system elements. Chapter 6 will cover the issues involved in designing operator-oriented human interface systems, including the low-level human interface device. Chapter 7 will address the same issues with regard to engineer-oriented devices.

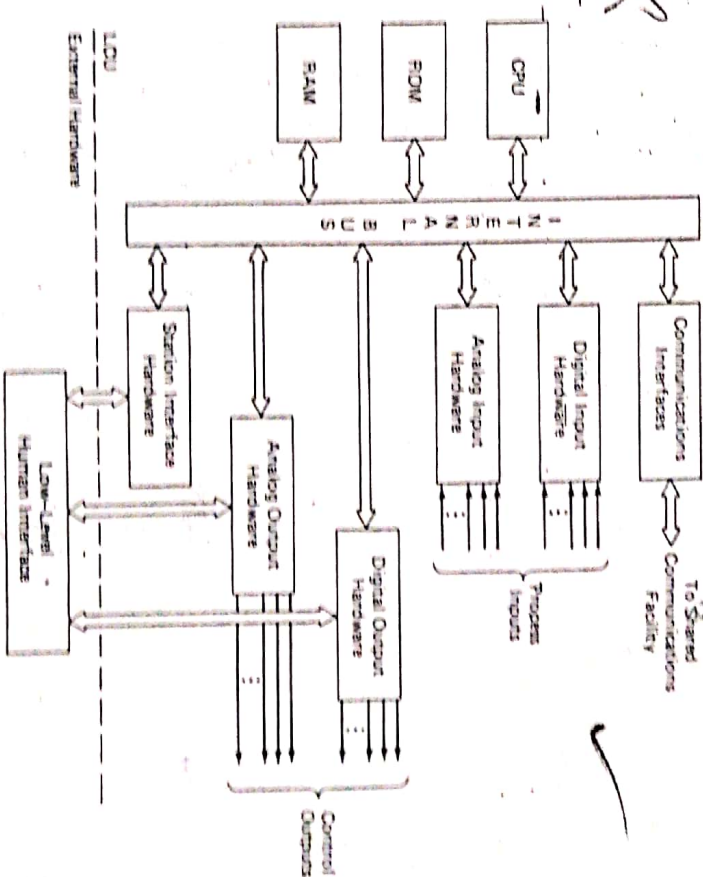


Figure 4.1. LCU Interfaces to Distributed System Elements

This chapter concentrates on process interfacing issues that relate to two specific areas: control system security and the process I/O capabilities of the distributed control system. Section 4.2 will discuss in depth the security issues, including the provision of manual override capability and redundant controllers. Then Section 4.3 will cover the issues involved in meeting a wide range of process interfacing requirements in a distributed system. These include the tradeoffs involved in using DI/OU's to augment the I/O capabilities of the LCU.

4.2. SECURITY DESIGN ISSUES FOR THE LOCAL CONTROL UNIT

4.2.1. Security Requirements

The first priority of the user of any process control system is to keep the process running under safe operating conditions. Downtime that curtails production is extremely expensive; an unsafe condition that leads to human injury or plant damage is even more costly.

Because of this, (reliability is one of the major factors considered in evaluating a distributed control system.) One way of designing a highly reliable control system is to manufacture it using only the highest-quality components, conduct extensive burn-in testing of the hardware, and implement other quality control measures in the production process. This will increase the mean time between failure (MTBF) of the system, and reduce the probability that it will cause a plant shutdown. However, every plant manager knows that any control system, no matter how reliable, will eventually fail. Therefore, it is important that the control system have adequate security features built into it so that the process can continue safely in spite of the failure of one of the elements of the control system. One can view the security objectives necessary in designing a distributed control system in the following hierarchy:

1. Maximize the availability of the *automatic* control functions of the system. As much as possible, make sure that the failure of a single control system element does not shut down all automatic control functions.
2. If the failure of a control system element causes the loss of automatic control in a portion of the system, make sure that there is a mechanism that allows the operator to take over manual control of that portion of the process.
3. As much as possible, ensure that the control outputs to the process are safe ones so that, if critical automatic and manual control functions are lost, the operator can shut the process down in an orderly and safe manner.

These security objectives are valid for logic control or sequential control subsystems as well as for continuous control. This hierarchy should be followed most carefully for critical control loops, such as those that control exothermic processes.

This section will discuss in detail a number of security features that can be built into distributed control systems to ensure that the above objectives are met. The manufacturers of these control systems have used a wide variety of approaches to designing security into their systems. Section 4.2.2 will give an overview of these approaches. The first step in triggering any security-related action is to detect that a failure in the control system has occurred. Section 4.2.3 will review the types of diagnostics used to detect such failures. Section 4.2.4 will discuss some of the design techniques used to ensure that the system generates only safe control outputs, even under failure conditions. Section 4.2.5 will cover the design of systems for secure manual backup of automatic control loops. Finally, Section 4.2.6 will summarize several approaches to using redundant hardware that permits automatic control functions to continue under failure conditions.

4.2.2. Overview of Security Design Approaches

Relatively few papers in the literature deal with the design of security features in distributed control systems. Some useful references in this area are given at the end of the chapter (Refs. 4.1-4.12). While each manufacturer of distributed systems takes a somewhat different approach to this design problem, there are three basic categories of security approaches currently in use. These are listed below in approximate order of increasing complexity and cost.

1. *Provide manual backup only (Figure 4.2A)*—In this case, each LCU is designed to implement only one or two control loops, and reliance is placed on the operator to take over manual control in case of a failure of the LCU. Note in the figure that the control output is fed back to the manual backup station and to the computation section of the controller, so that the inactive element can synchronize its output with the active element. This ensures that the output to the process will not be bumped when a switchover from the active to the inactive device occurs.
2. *Provide a standby redundant controller (Figure 4.2B)*—In this case, the LCU is backed up by another LCU that takes over if the primary controller fails. In this way, full automatic control is maintained even under failure conditions. As in the first case, the control output is fed back to both controllers to allow bumpless transfers to be accomplished.

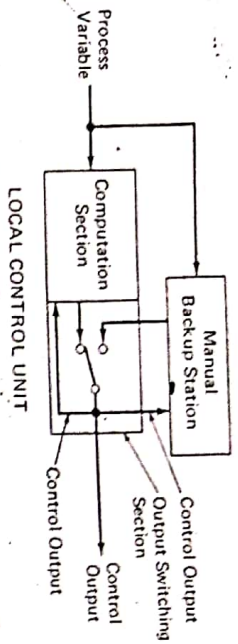


Figure 4.2A. Manual Backup Approach

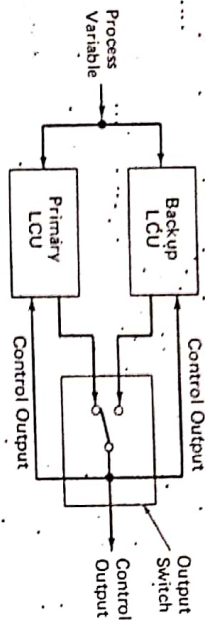


Figure 4.2B. Hot Standby Redundancy Approach

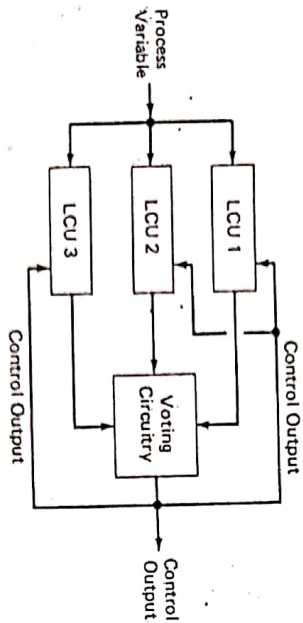


Figure 4.2C. Multiple Active Redundant Controllers

3. *Provide multiple active controllers (Figure 4.2C)*—In this case, several LCUs are active at the same time in reading process inputs, calculating control algorithms, and producing control outputs to the process. Since only one output can be used at a time, *voting* circuitry selects the valid output. The multiple active approach is designed so that a failure of one of the controllers does not affect the automatic control function. The selected control output is fed back so that each controller can compare its own output with the output generated by the voting device.

In each of these three approaches, the intent of the design is (1) to guarantee that multiple control channels (either manual or automatic) are able to generate the control output signal, and (2) to ensure that a safe channel is available or is switched in following the failure of one of the other channels.)

The manual backup approach relies on the ability of the operator to control the portion of the process associated with a single LCU. There is some argument on the maximum number of control outputs one operator can manipulate manually; however, handling one to four loops at one time is usually possible, the number depending on the speed of response required to keep each loop under control. This approach has its parallel with the security designs that discrete analog control systems provide, in which each loop is associated with a single physical controller and operator station. If the controller fails, only one loop is affected and the operator takes over manual control until a spare controller can be substituted. The single-loop integrity of this controller structure provides adequate security in the analog case; several manufacturers of distributed systems follow the same approach using microprocessor-based controllers. These controllers provide additional security through the "intelligence" of the microprocessor, which is capable of self-diagnosing potential or actual failures and generating safe control outputs when they occur.

In some situations, however, manual backup control alone does not provide an adequate level of security. This is the case when the LCU has to implement a larger number of control loops (say, five or more). It is unreasonable to expect an operator to handle all of these loops manually while the automatic controller is being repaired. The other situation occurs when the control loop is fast-acting, so that loss of automatic control for even a short time could cause an unsafe plant situation. In both of these cases, some form of redundant controller must be provided to carry on the automatic control functions in the event of a failure of the primary controller. The redundancy approach shown in Figure 4.2B relies on a "hot standby" controller to take over for the primary one. This approach has its roots in the direct digital control (DDC) computer systems described in Chapter 1. Because all of the plant control functions are implemented in a single DDC computer, a second computer to provide full backup of the primary computer is essential for control system security. The distributed control architecture uses other versions of backup, which will be described in Section 4.2.6.

The security design approach of using multiple active controllers to perform a control function had its origins in the "fly-by-wire" aircraft controllers developed in the early 1970s for supersonic transport and jumbo jets (see Reference 4.8, for example). These electronic controllers replaced

the physical cables the pilot used to manipulate the aircraft control surfaces. In this control application, a simple primary-plus-backup control architecture did not provide an adequate level of automatic control availability. Quadruple redundancy was necessary to provide a secure flight control system. As yet, this approach has not met widespread acceptance in the process control industries because of its high cost and complexity. However, it may become more feasible as hardware costs continue to decrease and specialized components are designed to simplify the system configuration.

4.2.3. On-Line Diagnostics

Every approach to providing control system security through backup elements requires that an on-line failure detection mechanism be included in the design of the LCU. Most distributed control system designs take advantage of the intelligence of the LCU's microprocessor by including a wide range of self-diagnostic routines in the repertoire of LCU functions. Discussed here are on-line routines used for the purpose of detecting impending or actual LCU failures. Section 7.4 will summarize mechanisms that relay this information to the operator.

Depending on the nature of the diagnostic test run and the time required for its execution, each test can be performed at startup of the LCU, during LCU operation, or on a periodic basis. In selecting the set of diagnostics to be used, the designer faces the limitations of memory and time needed to perform the basic LCU functions. The designer can allocate only a relatively small portion of the available resources to diagnostic routines without compromising the cost-effectiveness of the overall design of the LCU.

Table 4.1 lists types of diagnostics for detecting various kinds of failures. For each diagnostic test, the table gives a description, indicates a typical frequency of execution, and identifies the action taken by the LCU if the test is not accomplished successfully. No attempt will be made to describe these diagnostics in great detail since they are relatively standard tests used in systems other than distributed control. Most of them have been used in the past to improve the security of DDC computer applications and have been adapted for use in microprocessor-based systems. However, a few of them may need some further explanation. For example, the primary purpose of the analog-to-digital (A/D) converter check is to ensure that the converter is calibrated correctly (i.e., reads properly at the calibration voltages). Adding the process of detecting excessive calibration corrections makes the function a diagnostic one. The ROM/EAROM sum-

check is a method of verifying that the information stored in the memory at programming time has been retained without error. In the case of ROM, this sum is calculated and stored when the ROM is programmed in the factory. In the case of EAROM, EEPROM, and other alterable nonvolatile memory, the sum must be computed and stored periodically by the LCU processor itself, because the contents of memory change with time. The watchdog timer test provides a mechanism for shutting the LCU down that does not rely on the proper operation of the processor, as do most of the other diagnostics. The timer ensures that unsafe control outputs are not generated if the processor fails and loses track of its location in its program or halts completely.

Once a diagnostic test has detected a failure, the LCU must be able to act on this information in one or more of the following ways:

1. The LCU should be able to alarm or report the failure to both the low-level human interface and to the higher-level human interface and computing elements. If possible, both the *existence* and the *type* of failure should be reported.
2. The LCU should be able to switch a contact output to provide an external hardware indication of failure (in case the communication links between the LCU and the other elements are not operational).
3. If the failure affects only a portion of the LCU, the internal application logic of the LCU should be able to drive a failure indicator. The LCU's logic then can trigger the appropriate process trip sequence or cause the control logic to back off to a simpler level of control (e.g., manual instead of automatic, or simple loop instead of cascade.)
4. The LCU should be able to shut itself down in an orderly way, if necessary.

Any of these failure indications can trigger initiation of the manual backup function or transfer of control to a redundant controller.

4.2.4. Secure Control Output Design

This section suggests a number of design practices that can improve the security of the analog and digital control output hardware in an LCU. From a review of the LCU architecture in Figure 4.1, it is clear that the design of this hardware is critical to the security of the LCU. If this hardware is not well designed, incorporating security features into the rest of the system will not be able to compensate for the deficiencies in the output area. This holds for both analog and digital control output channels.

Table 4.1. On-Line LCU Failure Diagnostics

DIAGNOSTIC TYPE	NAME	DESCRIPTION	WHEN PERFORMED	ACTION ON FAILURE
Input diagnostics	A/D converter check	Processor applies known zero and span voltages to converter and uses measurements to correct for input errors.	Periodically during operation	If correction becomes too great, sets alarm and shuts converter down.
	Sensor out of range check	Processor checks that input from sensor is in acceptable range.	Every input scan	Declares sensor input to have bad quality.
	Excessive rate of change check	Processor checks that time rate of change of input from sensor is in acceptable range.	During operation	Declares sensor input to have bad quality.
	Open T/C detection	Processor checks thermocouple for open circuit using standard methods.	During operation	Declares T/C input to have bad quality.
Configuration diagnostics	I/O hardware check	Processor checks that selected I/O hardware options are present.	At startup	Alarms and shuts LCU down.
	Memory check	Processor checks that selected memory options are present.	At startup	Alarms and shuts LCU down.
Memory diagnostics	ROM/EAROM sumcheck	Processor compares the computed sum of the contents of memory with the pre-stored correct value.	Periodically during Operation	Alarms a ROM failure and shuts LCU down.
	RAM test	Processor writes a known pattern into RAM, then reads back and checks the results.	At startup	Alarms a RAM failure and shuts LCU down.
Output diagnostics	D/A converter check	Processor writes a known value to the D/A converter, reads it back through analog channel, and compares results.	Periodically during operation	If error becomes too great, sets alarm and shuts converter down.
	Output register check	Processor writes a known number to the D/A converter and reads digital value back to verify number.	During operation	Sets alarm and shuts converter down.
End-to-end processor/memory diagnostic	Test problem	Processor executes a test control or arithmetic algorithm, then compares results with prestored answer.	At startup	Sets alarm and shuts LCU down.
External hardware check	Watchdog timer	Processor sets an external timer periodically to confirm proper operation.	Periodically during operation	Timer hardware shuts LCU down.
Power system diagnostics	Voltage monitor	Processor uses external hardware to monitor the voltages generated by the LCU power supply.	Continuously during operation	Alarms power supply failure and shuts LCU down.

6 OPERATOR INTERFACES

6.1. INTRODUCTION

The control and communication equipment described in the previous chapters performs the bulk of the automated functions that are required for operating an industrial process. For this automated equipment to be used in a safe and effective manner, however, it is absolutely necessary to have a well-engineered human interface system to permit error-free interactions between the humans and the automated system. Two distinct groups of plant personnel interact with the control system on a regular basis:

1. *Instrumentation and control system engineers*—These people are responsible for setting up the control system initially and adjusting and maintaining it from time-to time afterwards;
2. *Plant operators*—These people are responsible for monitoring, supervising, and running the process through the control system during startup, operation, and shutdown conditions.

As the generalized distributed control system architecture in Figure 1.4 shows, a human interface capability can be provided at one or both of two levels:

1. Through a low-level human interface (LLHI) connected directly to the local control unit or data input/output unit (DI/OU) via dedicated cabling;
2. Through a high-level human interface (HLHI) connected to an LCU or DI/OU only through the shared communications Facility.

The low-level human interface equipment used in distributed control systems usually resembles the panelboard instrumentation (stations, indicators, and recorders) and tuning devices used in conventional electric analog control systems. The HLHI equipment makes maximum use of the latest display technology (e.g., CRTs or flat-panel displays) and peripheral devices (e.g., printers and magnetic storage) that are available

on the market; it is configured in a console arrangement that allows operator and engineer to be seated during use.

When it is included in the system configuration, the LLHI generally is located geographically close to (within 100–200 feet of) the LCU or DI/OU to which it is connected. On the other hand, the HLHI can be located anywhere in the plant, including the central control room. The needs of the application will determine whether the particular installation has one or both levels of interface. Figures 6.1, 6.2, and 6.3 show some examples of typical installations and their corresponding equipment configurations.

Figure 6.1 illustrates a relatively small and simple installation. A single LCU located in the plant equipment room (sometimes called the *relay room*) performs all of the required control functions. Low-level human interface units located in the equipment room and the plant control room provide the complete operator and instrument engineer interface for the control system. This type of equipment configuration is typical of a stand-alone control system for a small process or of a small digital control system installed in a plant controlled primarily with conventional electrical analog or pneumatic equipment.

Figure 6.2 shows a typical structure of a complete plantwide control system. Several LCUs are used to implement the functions required in controlling the process; therefore, the control is *functionally distributed*. However, the LCUs are all located in a central equipment room area, and so it is not a *geographically distributed* control system. Both high-level and low-level human interface devices are located in the control room area for operational purposes. Most of the operator control functions are performed using the high-level interface; the low-level interface is included in the configuration primarily to serve as a backup in case the high-level

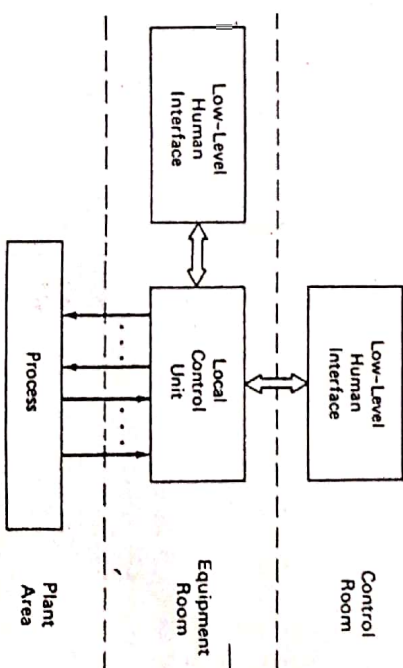


Figure 6.1. Stand-Alone Control Configuration

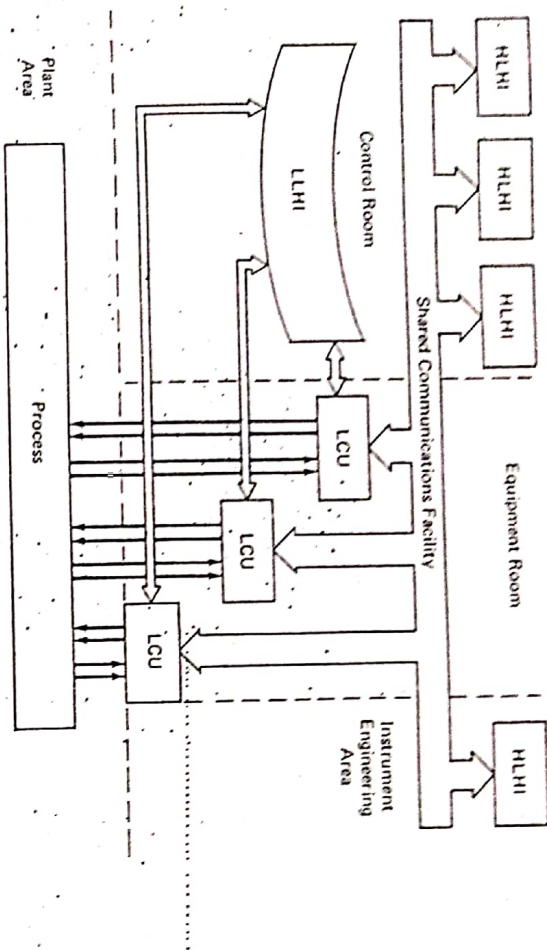


Figure 6.2. Geographically Centralized Control Configuration

interface fails. A high-level human interface is located in the instrument engineer's area so that control system monitoring and analysis can be done without disturbing plant operations. This type of installation is typical of early distributed control system configurations in which equipment location and operator interface design followed conventional practices.

Figure 6.3 shows a fully distributed control system configuration. In this case, each LCU is located in the plant area closest to the portion of the process that it controls. Associated low-level human interface equipment (if provided) is also located in this area. The control room and instrument engineering areas contain high-level human interface units, which are used to perform all of the primary operational and engineering functions. The low-level units are used only as manual backup controls in case the high-level equipment fails or needs maintenance. This configuration takes advantage of two areas of equipment savings that result from a totally distributed system architecture: (1) reduction in control room size (by eliminating panelboard equipment), and (2) reduction in field wiring costs (by placing LCUs near the process).

These examples of system configurations illustrate the point that human interface equipment in a distributed control system must be designed to meet a wide range of applications:

1. Large as well as small systems;
2. Centralized equipment configurations (often used in retrofit instal-

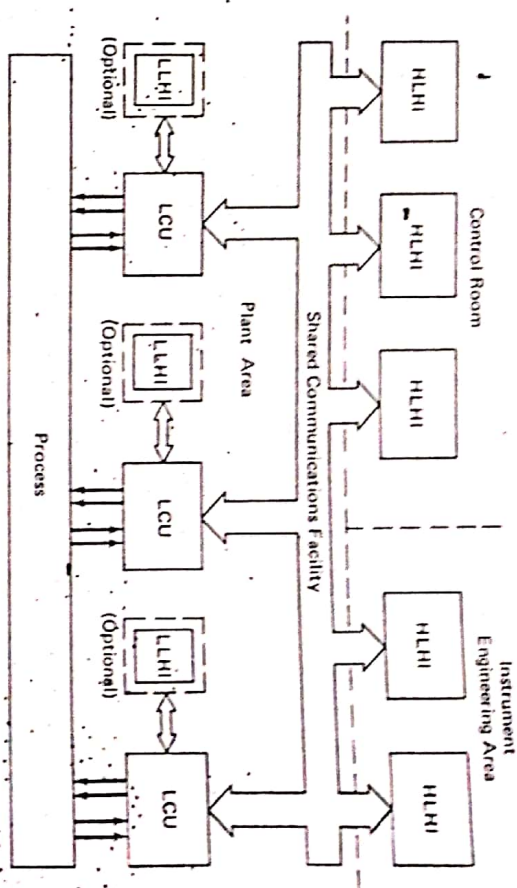


Figure 6.3. Geographically Distributed Control Configuration

lations made long *after* original plant construction) as well as distributed ones (likely in "grass roots" installations made *during* plant construction);

3. Variety of human interface philosophies, ranging from accepting CRT-only operation to requiring panelboard instrumentation in at least a backup role.

This chapter and the next provide an overview of the major issues to consider when evaluating or designing human interface equipment in a distributed control system. As in the previous chapters, the discussion will not be a detailed analysis but instead will address only the significant points; the references will permit the reader to go into any selected area in greater depth. This chapter discusses operator interface requirements and design issues; Chapter 7 will deal with instrument engineer interfaces. Section 6.2 summarizes the key requirements of an operator interface system; Sections 6.3 and 6.4 describe and evaluate alternative design approaches to implementing these requirements for the low-level and high-level operator interfaces, respectively.

6.2. OPERATOR INTERFACE REQUIREMENTS

Despite the continuing trend toward increased automation in process control and less reliance on the operator, the basic responsibilities of the operator have remained largely the same in the last fifty years. Most of

that are suitable for home or office environments may not be at all appropriate in a process plant or factory. Some of the requirements the designer should meet include the following:

1. Displays and pushbuttons should be sealed against the atmosphere to avoid contamination (from dirt or corrosive gases, for example);
2. Displays should be selected for high visibility in the expected ambient light environment;
3. Each pushbutton when depressed should provide tactile (touch) feedback to the operator to minimize potential errors.

A variety of display types has been developed for industrial use. The workhorse of the discrete display world is the light-emitting diode (LED). This component type is used in on/off status, bar-graph, and alphanumeric displays. It is quite suitable in most applications. However, bar-graph displays tend to be somewhat low in resolution when composed of LEDs rather than other display components. Gas-discharge and gas-plasma displays provide high-resolution and high-visibility bar graphs. The declining cost trend for these devices has made them an attractive display alternative to LEDs. Liquid crystal displays (LCDs) are very flexible and have been used in a wide variety of display configurations (mixtures of bar graphs, alphanumerics, and status displays). However, LCDs are not as visible as LEDs or gas-discharge displays, especially in low ambient lighting situations.

In the operator input area, the two most common types of push-button and switch inputs are:

1. *Spring-loaded plungers*—This component, often used in typewriter-like keyboards, can make use of a number of different switch-sensing mechanisms—magnetic, optical, or capacitive, for example. It is not used very often in panelboard equipment since its mechanical configuration makes it difficult to isolate the sensing mechanism from the environment.
2. *Membrane or dimple switches*—These are inexpensive mechanical switching components that come in a flat configuration suitable for direct mounting on a printed circuit board. An overlay sheet of mylar or plastic protects the switch assemblies from the environment. If packaged properly, they provide tactile feedback to the operator when depressed.

There are many other types of displays and switches suitable for use in panelboard equipment. For a survey of these types and their relative advantages and disadvantages, consult references such as 6.21 and 6.25.

Application in Distributed Systems. In the early installations of distributed control systems, both suppliers and users were concerned whether the operating personnel in the plants would accept the new technology. This involved two aspects of the new technology: (1) using microprocessors to implement closed-loop control functions, and (2) using new human interface hardware such as CRTs as the primary operating tool. As a result, those responsible for many of these installations took a "belt and suspenders" approach to the design of operator interface equipment by providing *both* panelboard instrumentation and CRT consoles. Operators were first trained to use the panelboard-type equipment, then were gradually weaned from it in favor of the CRT consoles. As the next section of this chapter will discuss, few industrial users now have any hesitation in providing CRT consoles as the primary operator interface tool. The LLOI equipment described in this section does, however, find continuing application in small control systems and as a backup mechanism for critical process control loops.

6.4. HIGH-LEVEL OPERATOR INTERFACE

In contrast with the low-level operator interface described in the previous section, the high-level operator interface in a distributed control system is a shared interface that is not dedicated to any particular LCU. Rather, the HLOI is used to monitor and control the operation of the process through any or all of the LCUs in the distributed system. Information passes between the HLOI and the LCUs by means of the shared communications facility, as described in Chapter 5.

While the LLOI hardware resembles conventional panelboard instrumentation, HLOI hardware uses CRT or similar advanced display technology in console configurations often called video display units (VDUs). The HLOI accepts operator inputs through keyboards instead of the switches, push-buttons, and potentiometers characteristic of conventional operator interface panels. Other digital hardware prints, stores, and manipulates information required in the operator interface system. This section will discuss many of the issues involved in the design of the HLOI. References 6.9–6.14 provide additional information on these issues.

In general, the use of microprocessor-based digital technology in the design of the HLOI system allows the development of a hardware configuration that departs radically from the design of panelboard-based operator interfaces. This configuration provides the user with several significant advantages over previous approaches:

1. Control room space is reduced significantly; one or a few VDUs can

replace panelboards several feet to 200 feet in length, saving floor space and equipment expense.

2. One can design the operator interface for a specific process plant in a much more flexible manner. The hardware operations peculiar to panelboard design and implementation (e.g., selecting control stations and cutting holes in panels) are eliminated. Instead, CRT display formats define the key operator interface mechanisms. One can change these formats during startup and duplicate selected information displays wherever necessary. Being able to do this leads to a much more usable operator interface configuration than conventional approaches.

3. Using microprocessors permits cost-effective implementation of functions that previously could be accomplished only with expensive computers. These include color graphic displays that mimic the organization of the process; information presented in engineering units; and advanced computing and data storage and retrieval functions.

However, one must take great care in designing the HLOI to achieve these benefits while minimizing any negative effects or concerns on the part of the operating personnel. It became evident during the early introduction of this technology to the marketplace that operators accept properly designed HLOIs very quickly. This is especially true of younger operators who have been preconditioned and pretrained by video games and personal computers.

6.4.1. Architectural Alternatives

All high-level operator interface units in distributed control systems are composed of similar elements: operator display; keyboard or other input device; main processor and memory; disk memory storage; interface to the shared communication facility; and hard-copy devices and other peripherals. However, the architectures of the various HLOIs on the market vary significantly depending on the way in which these common elements are structured.

Figure 6.6 shows one architecture commonly used in computer-based control systems and early distributed systems. In this architecture, there is a single central processing unit (and associated random-access memory) that performs all of the calculations, database management and transfer operations, and CRT-and-keyboard interfacing functions for the *entire* HLOI system. A separate communications controller interfaces the central processor with the shared communications facility.

There are several advantages to this configuration. First, there is a single

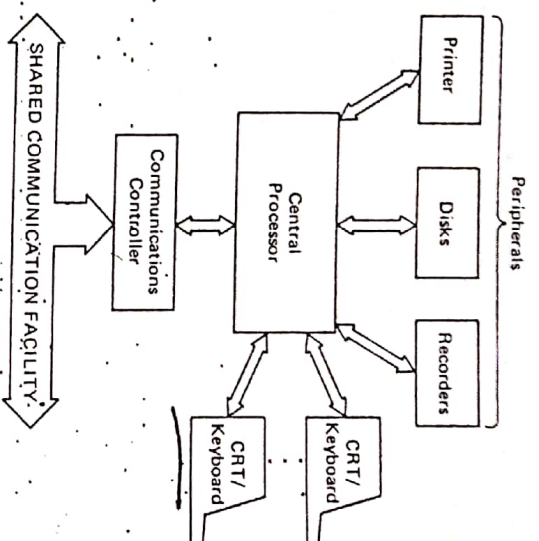


Figure 6.6. Centralized HLOI Configuration.

database of plant information that is updated from the communication system. As a result, each of the CRTs has access to any of the control loops or data points in the system. This is desirable, since it means that the CRTs are all redundant and can be used to back each other up in case of a failure. Another advantage is that the peripherals can be shared and need not be dedicated to any particular CRT/keyboard combination. This can reduce the number of peripherals required in some situations.

The disadvantages of this configuration are similar to those of all centralized computer systems:

1. It is an "all eggs in one basket" configuration, and so is vulnerable to single-point failures. In some cases, redundant elements can be provided; but this approach can lead to complex peripheral-switching and memory-sharing implementations.
2. Any single-processor, single-memory configuration has limitations on the number of loops and data points it can handle before its throughput or memory capacity runs out. In many operator interface systems of this type, the display response times are long and the size of system that can be handled is severely limited.
3. The centralized architecture is not easily scalable for cost-effective-ness: if it is designed properly to handle large systems, it may be too expensive for small ones.